

LokiGame App Summary

Generated from repository evidence only (code, config, and docs in this repo).

What It Is

LokiGame is a Spring Boot backend for a server-authoritative RPG/gacha-style game. It provides authenticated REST APIs for hero, dungeon, battle, inventory, and economy systems.

Who It Is For

Primary persona: game client/backend developers integrating and operating a live game API backend.

What It Does

- JWT auth APIs (register/login/refresh) with Spring Security filters and role checks.
- Hero workflows, including class/origin/world data and hero roll/management endpoints.
- Dungeon and dungeon-run orchestration, including seeds, scaling, and rewards paths.
- Battle simulation endpoints backed by battle service/engine classes.
- Inventory/equipment flows with generated equipment and owned inventory items.
- Economy/admin flows for currency request submission and review/approval endpoints.
- Session tracking hooks with Redis configured in application properties.

How It Works (Architecture)

- API layer: controllers under `src/main/java/.../controller` expose REST endpoints.
- Business layer: services under `.../service` implement game logic and orchestration.
- Core logic: generator (HeroFactory, EquipmentGenerator) and engine (BattleEngine).
- Data layer: Spring Data repositories persist entities to PostgreSQL (JPA/Hibernate).
- Security layer: SecurityConfig + JwtAuthenticationFilter + JwtTokenProvider.
- Startup/data flow: GameDataInitialize loads static game data from `resources/data/*.json`.
- Infra services: Redis configured; exact production cache/session strategy Not found in repo.

How To Run (Minimal)

1. In project root, run: `docker-compose up --build`
2. Wait for backend startup logs (service on port 8080).
3. Verify API: `http://localhost:8080/swagger-ui/index.html` or `/actuator/health`.
4. Alternative local run: set PostgreSQL/Redis env vars, then run `mvn spring-boot:run`.